

The Command-Line Surface

How typing a command in a terminal becomes a running node — `hathor_cli`, the subcommand dispatcher, and the `run_node` boot path.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 21 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

console_scripts · argparse · configargparse · dispatch table · structlog · Twisted reactor

Table of Contents

Chapter 21 — The Command-Line Surface	3
21.1 Localization — where hathor_cli/ sits	3
21.2 What it does and why it exists	4
How hathor-cli reaches Python — the console-script entry point	5
21.3 The concepts it rests on	5
21.4 The code, walked: dispatch	6
A toy dispatcher first	6
The real dispatch table: CliManager	7
Dispatch and the logging hooks: execute_from_command_line	8
The entry point: main()	9
21.5 The run_node boot path	10
Step 1 — parse arguments	10
Step 2 — select the network and load settings	11
Step 3 — assemble and start: prepare()	11
Step 4 — run the reactor	13
A worked illustration of reuse: quick_test	13
21.6 How it plugs into the lifecycle	13
Recap	14

Chapter 21 — The Command-Line Surface

What you'll learn in this chapter

- How typing `hathor-cli` in a terminal actually reaches Python code — the **console-script entry point** that the packaging system installs.
- How a single program with ~40 sub-tools routes the first word you type (`run_node`, `gen_peer_id`, `shell`, ...) to the right module, using a **dispatch table** instead of a chain of `if/elif`.
- The role of the two logging hooks (`PRE_SETUP_LOGGING`, `LOGGING_CAPTURE_STDOUT`) that let each subcommand opt into or out of the shared logging setup.
- The full `run_node` boot path in code: parse arguments → pick the network/settings → assemble the node with a builder → start it → hand control to the reactor.
- Where this chapter **hands off**: to settings (Ch 22), the reactor (Ch 23), and the composition root (Ch 24).

This is the first chapter of Part II, and so the first time the book opens real `hathor-core` source. We start here for one reason: this is the front door. Every other subsystem in the node is reached, ultimately, because someone typed a command and a dispatcher routed it. Chapter 0 told the story of "the life of a node" in prose; this chapter shows you the whole of Act I in code — the command is dispatched, arguments and logging are set up, settings are chosen, the node is assembled, started, and the reactor is run — and points you at the chapters that go deep on each later step.

We will lean on three ideas you have already met in Part I: the **dispatch table** (Ch 4), **CLI-as-configuration** with `argparse/configargparse` (Ch 19), and the **Twisted reactor** (Ch 16). This chapter does not re-teach them; it re-establishes each in context and points back.

21.1 Localization — where `hathor_cli/` sits

`hathor-core` is two top-level Python packages: `hathor/`, which is the node itself, and `hathor_cli/`, which is the thin command-line surface that launches and operates it. `hathor_cli/` is the outermost layer — it imports from `hathor/`, never the other way around.

```

hathor-core/
├── pyproject.toml           ← declares the `hathor-cli` console script
├── hathor_cli/             ← THE COMMAND-LINE SURFACE ◀ YOU ARE HERE
│   ├── main.py             ← CliManager: the dispatch table + entry point main()
│   ├── run_node.py         ← RunNode: the boot path for an actual node
│   ├── run_node_args.py    ← RunNodeArgs: the typed (Pydantic) model of run_node's flags
│   ├── util.py             ← create_parser(), setup_logging(), process_logging_*()
│   ├── quick_test.py       ← a RunNode subclass that exits after one tx
│   ├── side_dag.py         ← shell.py mining.py peer_id.py wallet.py ... (~40 tools)
│   └── ...
└── hathor/                ← the node (everything this CLI launches)
    ├── manager.py          ← HathorManager (Ch 29)
    ├── builder/            ← composition root (Ch 24)
    ├── reactor/            ← reactor abstraction (Ch 23)
    └── conf/               ← settings profiles (Ch 22)

```

The dependency arrow points one way: `hathor_cli/` reaches into `hathor/` (you will see it import `hathor.reactor`, `hathor.builder`, `hathor.conf` in §21.5), and `hathor/` never reaches back. That is deliberate. The node's logic should not know or care whether it was started from a terminal, a test harness, or a simulator. Keeping the entry point as a separate, dependent layer is what makes the same node assemblable three different ways.

Context — the front door. `hathor_cli/` is where a human (an operator, a developer, a miner) meets the node. It owns nothing about the ledger, the network, or consensus. Its single job is to turn a line of text typed in a terminal into the right call into `hathor/`. Because it is the entry point, it is also where two cross-cutting concerns are set up first, before anything else runs: command-line **arguments** and **logging**. Everything downstream assumes those are already in place.

21.2 What it does and why it exists

A full node is not the only program a Hathor operator runs. They also need to generate a peer identity, create a wallet, run a miner, export the database, dump nano-contract state, open a debugging shell, and a couple dozen other small tasks. You could ship each of those as its own separate executable — `hathor-run-node`, `hathor-gen-peer-id`, `hathor-shell`, and so on — but that is forty commands for a user to remember and forty entries to install.

`hathor_cli/` takes the other approach, the same one `git` takes: **one program, many subcommands**. You type the program name, then the subcommand as the first argument:

```
hathor-cli run_node --testnet --data ./data
hathor-cli gen_peer_id
hathor-cli shell
```

The program reads that first word, finds the code responsible for it, and runs it. This is the pattern Chapter 0 §0.3 called "the command is dispatched" — Act I, step 1.

How `hathor-cli` reaches Python — the console-script entry point

When you `pip install` (or `poetry install`) `hathor-core`, how does the shell command `hathor-cli` come to exist? It is declared in `pyproject.toml`¹ (`pyproject.toml:37-38`):

```
[tool.poetry.scripts]
hathor-cli = 'hathor_cli.main:main'
```

This is a **console-script entry point**². At install time, the packaging tool reads that line and generates a tiny executable named `hathor-cli` on your `PATH`. That generated stub does essentially one thing: import the object named on the right — the function `main` in the module `hathor_cli.main` — and call it. The format is always `package.module:object`. So typing `hathor-cli` in a terminal is, after one layer of generated glue, exactly the same as running `from hathor_cli.main import main; main()`.

Note that the packages list (`pyproject.toml:32-35`) ships both `hathor` and `hathor_cli` as installable packages. You will sometimes see the codebase import a CLI module as `hathor.cli.*` and sometimes as `hathor_cli.*`; both spellings appear in this very file's source. The canonical entry point, the one packaged into the executable, is `hathor_cli.main:main`. When you `grep` for CLI code, search for both spellings to be safe.

21.3 The concepts it rests on

This chapter sits on three Part I ideas. Each gets a one-paragraph re-establishment here and a pointer to its full treatment — do not skip these; meeting an idea again in context is how it sticks.

Recap — the dispatch table (full treatment in Ch. 4). A dispatch table is a dictionary that maps a key (here, a command name like `"run_node"`) to the thing that handles it (here, a Python module). To "dispatch" is to look up the handler by key and invoke it. The alternative is an `if cmd == "run_node": ... elif cmd == "shell": ...` ladder. The dictionary wins on three counts: adding a command is one line, not a new branch; the set of commands becomes data you can iterate (to print a help screen); and lookup is $O(1)$ regardless of how many commands there are. The cost is one layer of indirection — you can't read the control flow top-to-bottom. For ~40 commands, that trade is clearly worth it. → Ch 4.

Recap — CLI as configuration: argparse and configargparse (full treatment in Ch. 19). `argparse` is Python's standard library for declaring command-line flags (`--testnet`, `--data PATH`) and parsing them into a structured object. `configargparse` is a drop-in superset that adds one capability: a flag can be supplied either on the command line or from an environment variable or from a config file. A node is run by operators and by automated deployments, so being able to set the same option three ways matters. `hathor_cli` builds every parser through one helper, `create_parser()` in `hathor_cli/util.py:30`, which returns a `configargparse.ArgumentParser` with the env-var prefix `hathor_`. → Ch 19.

Recap — the Twisted reactor (full treatment in Ch. 16, recap in Ch. 23). Twisted is the asynchronous framework `hathor-core` is built on. Its centerpiece is the **reactor**: a single event loop that, once running, waits for events (a peer connects, a timer fires, data arrives) and calls the right handler for each. The crucial fact for this chapter: the reactor is created early but only runs at the very end of boot. `reactor.run()` is a blocking call — it does not return until the node shuts down. So the last line of the `run_node` boot path is, in effect, "hand the program over to Twisted forever." → Ch 16 & 23.

There is also one component this chapter points at but does not open: the **composition root**, Hathor's two builders (`Builder` and `CliBuilder`). The `run_node` path uses `CliBuilder` to assemble the fully-wired node before starting it. That assembly is intricate enough to get its own chapter. → Ch 24.

21.4 The code, walked: dispatch

A toy dispatcher first

Before the real `CliManager`, here is the whole idea in five lines of plain Python:

```
def run_node(): print("booting a node...")
def gen_peer_id(): print("new peer id...")

COMMANDS = {"run_node": run_node, "gen_peer_id": gen_peer_id} # the table

def main(argv):
    cmd = argv[1] # the first word after the program name
    COMMANDS[cmd]() # look it up, call it – that's dispatch
```

`COMMANDS` is the dispatch table. `main` reads the first argument, looks up the handler, and calls it. Adding a third command is one new dictionary entry. The real Hathor code is this idea, hardened: the values are modules (not functions), there is grouping for the help screen, there is error handling for unknown commands, and there is a logging-setup step wrapped around the call.

The real dispatch table: `CliManager`

The dispatcher is the class `CliManager` in `hathor_cli/main.py:26`. Its constructor builds the table. First it imports every subcommand module (`hathor_cli/main.py:34-68`), then registers each one with `add_cmd` (starting at `hathor_cli/main.py:70`):

```
self.add_cmd('hathor', 'run_node', run_node, 'Run a node')
self.add_cmd('hathor', 'gen_peer_id', peer_id, 'Generate a new random peer-id')
self.add_cmd('mining', 'run_miner', mining, 'Run a mining process (running node required)')
self.add_cmd('dev', 'shell', shell, 'Run a Python shell')
self.add_cmd('multisig', 'gen_multisig_address', multisig_address, 'Generate a new multisig address')
# ... ~40 of these, grouped: hathor / mining / side-dag / docs / multisig /
# tests / wallet / oracle / events / features / dev
```

`add_cmd` (`hathor_cli/main.py:110`) does the bookkeeping behind the table:

```
def add_cmd(self, group, cmd, module, short_description=None):
    self.command_list[cmd] = module # the dispatch table itself
    self.groups[group].append(cmd) # grouping, for the help screen
    if short_description:
        self.cmd_description[cmd] = short_description
    self.longest_cmd = max(self.longest_cmd, len(cmd)) # for aligned help output
```

Two things to notice. First, `command_list` (`hathor_cli/main.py:29`) is the dispatch table: command-name → module. Second, the value is a whole module (`ModuleType`), not a function. The convention across `hathor_cli` is that every subcommand module exposes a top-level `main()` function; dispatching a command means "call `module.main()`." Storing the module rather than `module.main` lets the dispatcher also read other top-level attributes off it — which is exactly how the logging hooks work (below).

The grouping (`self.groups`, a `defaultdict(list)` at `hathor_cli/main.py:31`) exists only to produce a readable help screen. `help()` (`hathor_cli/main.py:117`) sorts the groups and prints each command under its group heading, padded to `longest_cmd` so the

descriptions line up. This is the payoff of making the command set data: the help screen is generated by iterating the same structure that drives dispatch, so the two can never drift out of sync.

Dispatch and the logging hooks: `execute_from_command_line`

The actual routing happens in `execute_from_command_line` (`hathor_cli/main.py:134`).

Walk it top to bottom:

```
def execute_from_command_line(self):
    from hathor_cli.util import process_logging_options, process_logging_output, setup_logging

    if len(sys.argv) < 2:          # no subcommand given → print help, exit 0
        self.help()
        return 0

    cmd = sys.argv.pop(1)          # take the first word; REMOVE it from argv
    if cmd == 'help':
        self.help()
        return 0

    if cmd not in self.command_list: # unknown command → friendly error, exit -1
        print('Unknown command: "{}"'.format(cmd))
        print('Type "{} help" for usage.'.format(self.basename))
        return -1

    sys.argv[0] = '{} {}'.format(sys.argv[0], cmd) # so the subcommand sees a sane progname
    module = self.command_list[cmd]               # ← the dispatch lookup
```

The detail that trips people up is `sys.argv.pop(1)` (`hathor_cli/main.py:141`). The dispatcher consumes the subcommand word and removes it from `sys.argv`. After that line, `sys.argv` looks as if the subcommand module had been invoked directly — its own argument parser (which knows nothing about subcommands) sees only its flags. This is the standard `git`-style trick: the outer layer eats the first word, the inner layer parses the rest.

Now the two logging hooks. Each subcommand module may declare two optional top-level attributes; the dispatcher reads them off the module with `getattr` and a default:

```
if '--help' in sys.argv:
    capture_stdout = False
else:
    capture_stdout = getattr(module, 'LOGGING_CAPTURE_STDOUT', False) # main.py:157
# ... (a --pdb debugger hook omitted) ...
pre_setup_logging = getattr(module, 'PRE_SETUP_LOGGING', True)       # main.py:167
if pre_setup_logging:
    output = process_logging_output(sys.argv)
    options = process_logging_options(sys.argv)
    setup_logging(logging_output=output, logging_options=options, capture_stdout=capture_stdout)
    module.main()
else:
    module.main(capture_stdout=capture_stdout)
```


This is the wrapper pattern from Chapter 4 applied to logging: the dispatcher wraps each subcommand call with logging setup, and the subcommand customizes that wrapper by setting two flags.

- **PRE_SETUP_LOGGING** (default `True`) decides who sets up structured logging³. If `True`, the dispatcher does it — it calls `setup_logging` before calling the command. If a module sets it to `False`, the dispatcher does **not** set up logging itself; instead it passes `capture_stdout` into the command's own `main()`, trusting the command to configure logging on its own schedule. A command that needs to read some of its own arguments before configuring logging would set this `False`.
- **LOGGING_CAPTURE_STDOUT** (default `False`) decides whether plain `print()` output should be redirected into the structured logging stream. It is forced `False` whenever `--help` is present (`hathor_cli/main.py:154`), because help text must go straight to the terminal, un-captured.

Both are read with `getattr(module, NAME, default)`, so a subcommand module that declares neither attribute gets the defaults (`PRE_SETUP_LOGGING=True`, `LOGGING_CAPTURE_STDOUT=False`): logging is set up for it, stdout is not captured. The opt-in is per-command, expressed as data on the module, not as special cases inside the dispatcher. The dispatcher stays generic.

`process_logging_output` (`hathor_cli/util.py:138`), `process_logging_options` (`hathor_cli/util.py:159`), and `setup_logging` (`hathor_cli/util.py:172`) all live in `hathor_cli/util.py`. The first two peel the logging flags (`--json-logs`, `--disable-logs`, `--debug`) out of `sys.argv` before the main parser runs and return small typed records (`LoggingOutput`, `LoggingOptions`); `setup_logging` then configures structlog from them — including an observer that funnels Twisted's own log messages into the same stream (`hathor_cli/util.py:355`). The canonical treatment of structured logging is Chapter 17.

The entry point: `main()`

The function the console script actually calls is `main` at the bottom of the file (`hathor_cli/main.py:177`):

```
def main():
    try:
        sys.exit(CliManager().execute_from_command_line())
    except KeyboardInterrupt:
        logger.warn('Aborting and exiting...')
        sys.exit(1)
    except Exception:
        logger.exception('Uncaught exception:')
        sys.exit(2)
```

Three things happen here. It constructs a fresh `CliManager` (building the dispatch table), runs `execute_from_command_line`, and passes that return value to `sys.exit` — so the integer a subcommand returns becomes the process's **exit code**⁴. The `try/except` is the program's last line of defence: a `Ctrl-C` becomes a clean "Aborting..." and exit code 1; any other uncaught exception is logged with a full traceback (`logger.exception`) and exits with code 2. This is why an operator who kills a node sees a tidy message rather than a Python traceback, and why a crash always produces a logged stack trace and a non-zero exit code a deployment system can detect.

21.5 The `run_node` boot path

Dispatch ends with `module.main()`. For the `run_node` subcommand, that module is `hathor_cli/run_node.py`, whose `main()` is two lines (`hathor_cli/run_node.py:595`):

```
def main():
    RunNode().run()
```

All the work is in the class `RunNode` (`hathor_cli/run_node.py:49`). Crucially, almost everything happens in its **constructor** — `RunNode()` boots the entire node — and `run()` only starts the reactor at the very end. We walk the constructor (`__init__`, `hathor_cli/run_node.py:501`) in order.

Step 1 — parse arguments

The constructor builds the parser and parses `argv` (`hathor_cli/run_node.py:514-515`):

```
self.parser = self.create_parser()
raw_args = self.parse_args(argv)
self._args = self._parse_args_obj(vars(raw_args))
```

`create_parser` (a classmethod, `hathor_cli/run_node.py:64`) starts from `create_parser()` in `util.py` (the configargparse parser) and adds every `run_node` flag — `--testnet`, `--data`, `--listen`, `--stratum`, `--wallet`, and dozens more (`hathor_cli/run_node.py:75-177`). Network flags are grouped as mutually exclusive (`hathor_cli/run_node.py:80`) so you cannot pass `--testnet` and `--localnet` together.

`_parse_args_obj` (`hathor_cli/run_node.py:587`) then validates the raw dict into a typed model:

```
def _parse_args_obj(self, args):
    from hathor_cli.run_node_args import RunNodeArgs
    return RunNodeArgs.model_validate(args)
```

Recap — RunNodeArgs , the typed flag model (Pydantic full treatment in Ch. 18, settings in Ch. 22). `RunNodeArgs` (`hathor_cli/run_node_args.py:20`) is a Pydantic model: a class whose fields are typed (`hostname: str | None` , `auto_hostname: bool` , ...) and validated at construction. It is configured `extra='forbid'` , `frozen=True` (`run_node_args.py:21`), meaning an unexpected key is an error and the object is immutable once built. Parsing the argv dict through this model converts a loose bag of strings into a checked, read-only configuration object the rest of boot can trust. The class doc-comment in `create_parser` even reminds maintainers that any new CLI flag must be added here too (`run_node.py:67-68`). → Ch 18, Ch 22.

Step 2 — select the network and load settings

Still in the constructor, the parsed args choose which network the node joins. The mechanism is an environment variable that points at a settings YAML file (`hathor_cli/run_node.py:519-532`):

```
if self._args.config_yaml:
    os.environ['HATHOR_CONFIG_YAML'] = self._args.config_yaml
elif self._args.testnet:
    os.environ['HATHOR_CONFIG_YAML'] = TESTNET_INDIA_SETTINGS_FILEPATH
elif self._args.nano_testnet:
    os.environ['HATHOR_CONFIG_YAML'] = NANO_TESTNET_SETTINGS_FILEPATH
elif self._args.localnet:
    os.environ['HATHOR_CONFIG_YAML'] = LOCALNET_SETTINGS_FILEPATH
```

This is the single most consequential choice at boot: get the network wrong and the node speaks a different dialect than its peers and will never sync. (Note that some retired networks — `testnet-hotel` , `testnet-golf` — are now hard rejections, `run_node.py:523-528` .) Immediately after, the constructor forces the settings to load and validates them early (`hathor_cli/run_node.py:534-540`):

```
try:
    get_global_settings()
except (TypeError, ValidationError) as e:
    raise PreInitializationError('An error was found while trying to initialize HathorSettings...')
```

Loading settings here, before any heavy assembly, means a malformed profile fails fast with a clear error rather than halfway through building the node. The settings machinery — how `HATHOR_CONFIG_YAML` becomes a frozen `HathorSettings` object — is Chapter 22.

Step 3 — assemble and start: `prepare()`

The constructor's last real act is to call `self.prepare()` (`hathor_cli/run_node.py:542`). `prepare()` (`hathor_cli/run_node.py:180`) is where the node comes into being. In order:

Process hygiene and safety gates. It sets the process title (`setproctitle` , `run_node.py:185`), the recursion limit (`run_node.py:187-190`), and checks the open-file-descriptor limit is at least 256, exiting if not (`run_node.py:192-195`). Then three guards run: `validate_args` , `check_unsafe_arguments` , and `check_python_version` (`run_node.py:197-199`). The unsafe-arguments guard (`run_node.py:387`) is worth knowing: certain flags — listed in `UNSAFE_ARGUMENTS` (`run_node.py:50-60`), e.g. `--test-mode-tx-weight` , `--x-asyncio-reactor` — are refused unless `--unsafe-mode` is explicitly given, so a dangerous testing flag cannot be left on against mainnet by accident.

Create the reactor (but do not run it). (`hathor_cli/run_node.py:201-203`):

```
from hathor.reactor import initialize_global_reactor
reactor = initialize_global_reactor(use_asyncio_reactor=self._args.x_asyncio_reactor)
self.reactor = reactor
```

The reactor is created now and stored, but its event loop is not started until step 4. By default this is the Twisted reactor; `--x-asyncio-reactor` swaps in an asyncio-backed one (the abstraction that makes that swap possible is Chapter 23).

Assemble the node with `CliBuilder`. (`hathor_cli/run_node.py:205-213`):

```
from hathor_cli.builder import CliBuilder
builder = CliBuilder(self._args)
try:
    self.manager = builder.create_manager(reactor)
except BuilderError as err:
    self.log.error(str(err))
    sys.exit(2)
```

This is Chapter 0 §0.3 Act I step 5. `CliBuilder` constructs, in dependency order, every component the node needs — storage, indexes, wallet, pub-sub and the event manager, verification, consensus, the nano-contract runtime, feature activation — and wires them into a single `HathorManager` (`hathor_cli/run_node.py:210`). A failure during assembly is caught and turned into a clean log line plus exit code 2, rather than a raw traceback. The builder is Chapter 24; the manager it produces is Chapter 29.

Wire up listeners and resources. With the manager built, `prepare()` attaches the optional network-facing servers to the reactor: the Stratum mining server (`run_node.py:218-228`) and, via a `ResourcesBuilder` , the HTTP status/API server (`run_node.py:233-248`). Each is bound with `reactor.listenTCP(...)` — registered with the reactor, but nothing flows until the loop runs.

Start the manager. (`hathor_cli/run_node.py:250`) calls `self.start_manager()` , which (`run_node.py:292-294`) starts Sentry if configured and then calls `self.manager.start()` . This is Chapter 0 §0.3 Act I step 6: the crash check, the genesis load and in-memory rebuild, and the bringing-online of subsystems (WebSocket, metrics, the P2P connections manager, wallet). The mechanics live in Chapter 29.

After `prepare()` returns, the constructor registers OS signal handlers (`run_node.py:543`; `SIGUSR1` reloads peers, `SIGUSR2` opens a `sysctl` pipe) and, if `--sysctl` was given, starts the runtime-control socket (`run_node.py:544-545`). The node is now fully built and started — but still frozen, because the reactor has not run.

Step 4 — run the reactor

Back in `main()`, after the constructor returns, `RunNode.run()` is called (`hathor_cli/run_node.py:591`):

```
def run(self):
    self.reactor.run()
```

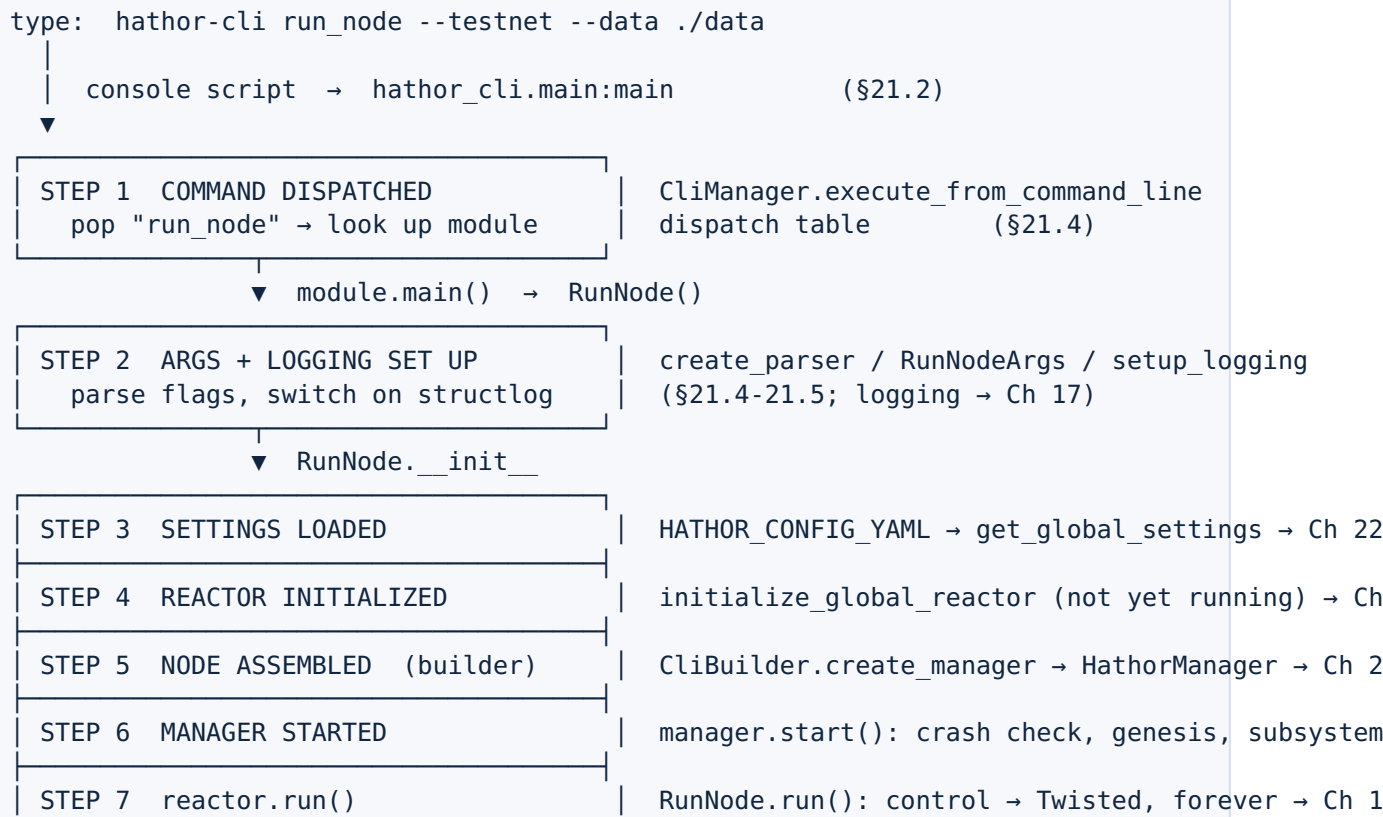
This is Chapter 0 §0.3 Act I step 7 — "the reactor is put in gear." `reactor.run()` blocks; control passes to Twisted's event loop and does not return until shutdown. From here the node is reactive: it sleeps until an event arrives (a peer connects, a vertex is received, a timer fires) and dispatches the matching handler. → Ch 16 & 23.

A worked illustration of reuse: `quick_test`

`quick_test` (`hathor_cli/quick_test.py`) is a subclass of `RunNode` that boots a node exactly as above but stops after it receives a single transaction — a smoke test. That it is a subclass (inheritance, Ch 1) rather than a copy is the reason a one-off test command stays in lock-step with the real boot path: change `RunNode`, and `quick_test` inherits the change. This is the same payoff as the dispatch table — write the boot sequence once, vary only what differs.

21.6 How it plugs into the lifecycle

Put against the life-of-a-node story (Chapter 0 §0.3, Act I), this chapter owns the whole startup sequence — but implements only its first two steps, delegating the rest:



Steps 1-2 are `hathon_cli`'s own job, done in code you have now read in full (`main.py`) and walked line by line (`run_node.py`). Steps 3 onward are invoked from `run_node.py` but implemented elsewhere, and each is a chapter of its own. The CLI's contribution is to be the reliable, generic front door: route the command, parse and validate the configuration, choose the network, turn on logging, then drive a builder and start the manager — and finally get out of the way by handing the process to the reactor.

Recap

Phase	File (<code>hathon_cli/...</code>)	Responsibility
Reach Python	<code>pyproject.toml:37-38</code>	<code>hathon-cli = 'hathon_cli.main:main' — console script → <code>main()</code></code>
Build the table	<code>main.py:26</code> <code>CliManager.__init__</code> , <code>add_cmd (:110)</code>	register ~40 subcommands → modules, grouped
Dispatch	<code>main.py:134</code> <code>execute_from_command_line</code>	pop first word, look up module, call <code>module.main()</code>

Phase	File (<code>hathor cli/...</code>)	Responsibility
Logging hooks	<code>main.py:157,167</code> <code>LOGGING_CAPTURE_STDOUT</code> , <code>PRE_SETUP_LOGGING</code>	per-command opt-in to shared logging setup
Entry + safety	<code>main.py:177</code> <code>main()</code>	construct manager, run, <code>sys.exit</code> , catch <code>Ctrl-C</code> / crashes
Boot a node	<code>run_node.py:49</code> <code>RunNode</code> , :595 <code>main()</code>	the whole <code>run_node</code> sequence
Parse + type args	<code>run_node.py:514-517</code> , <code>run_node_args.py:20</code>	<code>configargparse</code> parser → frozen Pydantic <code>RunNodeArgs</code>
Pick network	<code>run_node.py:519-540</code>	set <code>HATHOR_CONFIG_YAML</code> , load + validate settings early
Assemble + start	<code>run_node.py:180</code> <code>prepare()</code> (<code>reactor</code> :202 , <code>CliBuilder</code> :208 , <code>start_manager</code> :250)	build → wire → start the manager
Run reactor	<code>run_node.py:591</code> <code>run()</code> → <code>reactor.run()</code>	control → Twisted, until shutdown
Parser + logging	<code>util.py:30</code> <code>create_parser</code> , :172 <code>setup_logging</code>	<code>configargparse</code> parser; structlog configuration

The command-line surface is small and deliberately dumb: it holds no ledger logic, only the routing and configuration plumbing that every subcommand needs before its real work begins. The two ideas to carry forward are the **dispatch table** (the command set is data, so adding a command and generating the help screen are both one-liners) and the **wrapped call** (the dispatcher sets up logging around each subcommand, customized by two flags the module declares). For `run_node` , the chapter's parting picture is the seven-step boot path — and the next two chapters fill in its first hand-offs: **Chapter 22** explains how step 3 turns `HATHOR_CONFIG_YAML` into a frozen settings object, and **Chapter 24** explains how step 5's `CliBuilder` assembles the wired node that step 6 then starts.

-
1. `pyproject.toml` is the standard configuration file for a modern Python project — dependencies, build settings, and tool config in one place. Hathor uses Poetry to read it. Full treatment in Chapter 13. ↩

2. A console-script entry point is a line in packaging metadata of the form `name = "package.module:function"`. At install time the packaging tool generates an executable called `name` on your `PATH` that imports and calls that function. It is how a `pip install` makes a shell command appear. ↵
3. Structured logging records each log entry as machine-readable key-value data (e.g. `event="node started" network="testnet"`) rather than a freeform sentence, so logs can be searched and analyzed programmatically. Hathor uses the `structlog` library; full treatment in Chapter 17. ↵
4. An exit code (or status code) is the small integer a process returns to the shell when it ends. `0` means success; any non-zero value signals an error. Shells and deployment tools branch on it (`if program; then ...`). Hathor uses `0` for success, `1` for a `Ctrl-C` abort, `2` for an uncaught crash. ↵